LAB 05 Navigation, routes and passing data between screens

Unit 2 • 3-hour guided laboratory • CLO-5

Learning goal. Implement forward navigation, detail screens, reverse navigation with values and named routes.

Core concepts. Navigator stack (push/pop), route arguments, MaterialPageRoute, mounted/context safety, user-flow planning.

Before you start. Students know StatelessWidget and StatefulWidget; every task is an independent app.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab05
cd mad_lab05
```

Solved task 1 | Course list → course detail → saved selection

Problem and objective. Tap a course to navigate to its details, then return the selected course name as a value to the original screen.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: CoursesPage()));

class CoursesPage extends StatefulWidget {
  const CoursesPage({super.key});
  @override
  State<CoursesPage> createState() => _CoursesPageState();
}

class _CoursesPageState extends State<CoursesPage> {
  String selected = 'No course selected';
  final courses = const ['Mobile Application Development',
    'Database Systems', 'Software Engineering'];

  Future<void> openDetail(String course) async {
    final result = await Navigator.push<String>(context,
      MaterialPageRoute(builder: (_) => DetailPage(name: course)));
    if (!mounted || result == null) return;
    setState(() => selected = result);
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Course navigator')),
    body: Column(children: [
      Padding(padding: const EdgeInsets.all(16),
        child: Text('Chosen: $selected')),
      Expanded(child: ListView(
        children: courses.map((name) => ListTile(
          title: Text(name), trailing: const Icon(Icons.chevron_right),
          onTap: () => openDetail(name),
        )).toList(),
      )),
    ]),
  );
}

class DetailPage extends StatelessWidget {
  final String name;
  const DetailPage({super.key, required this.name});

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Course details')),
    body: Center(child: Column(mainAxisSize: MainAxisSize.min,
      children: [
```

```
        Text(name, style: Theme.of(context).textTheme.titleLarge),
        const SizedBox(height: 18),
        FilledButton(
          onPressed: () => Navigator.pop(context, name),
          child: const Text('Choose course'),
        ),
      ],
    )),
  );
}
```

How the solution works. push places a detail route on the navigation stack and yields a Future. pop(context, name) removes that route and resolves the Future with the chosen item. The mounted check prevents state changes after the original page has been disposed.

Expected output. Selecting a course opens a new page; pressing Choose course returns and updates the first page.

Check your work. Press Android Back without choosing and explain why the original selection remains unchanged.

Solved task 2 | Named routes with typed arguments

Problem and objective. Open a profile screen via a named route and pass a simple profile title in RouteSettings.arguments.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(MaterialApp(
  initialRoute: '/',
  onGenerateRoute: (settings) {
    if (settings.name == '/') {
      return MaterialPageRoute(builder: (_) => const HomePage());
    }
    if (settings.name == '/profile') {
      final label = settings.arguments as String? ?? 'Guest';
      return MaterialPageRoute(
        builder: (_) => ProfilePage(label: label), settings: settings);
    }
    return MaterialPageRoute(builder: (_) => const ErrorPage());
  },
));

class HomePage extends StatelessWidget {
  const HomePage({super.key});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Named routes')),
    body: Center(child: FilledButton(
      onPressed: () => Navigator.pushNamed(context, '/profile',
          arguments: 'Student representative'),
      child: const Text('Open profile'),
    )),
  );
}

class ProfilePage extends StatelessWidget {
  final String label;
  const ProfilePage({super.key, required this.label});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Profile')),
    body: Center(child: Text(label)),
  );
}

class ErrorPage extends StatelessWidget {
  const ErrorPage({super.key});
  @override
  Widget build(BuildContext context) => const Scaffold(
    body: Center(child: Text('Route not found')),
  );
}
```

How the solution works. onGenerateRoute centralizes name-to-page mapping and reads RouteSettings.arguments. The fallback route avoids silent failure for an unknown path. Type casting is guarded with as String? for null, but a wrong non-String type would still throw; typed route objects scale better in large apps.

Expected output. The Home screen navigates to a profile labeled Student representative; the app bar back arrow returns home.

Check your work. Change /profile to /missing and confirm the fallback screen renders.

Scenario-based practice — unsolved

Scenario 1. Exam workflow. Model a three-screen exam flow: instructions, timed questions and results. Decide when the back button is allowed and how an incomplete attempt should be handled.

Scenario 2. Order details. Navigate from orders to an order detail and back with an edited delivery note; define type-safe data transfer and cancellation behavior.

Technical viva — justify your reasoning

1. What is the difference between pushReplacement and push?
2. How does Navigator.pop return a typed value?
3. Why does a Future from push complete after the route is popped?
4. What happens to state if a widget is unmounted during an awaited navigation?
5. What risk follows casting route arguments without validation?
6. How would deep links change your route architecture?

Submission and self-check

Demonstrate push, pop, cancellation and unknown-route behavior; attach a two-screen navigation diagram.